


Controllo dei tipi

Linguaggi e tipi

Il problema che ci poniamo ora è quello di comprendere come la struttura dei tipi del linguaggio possa influenzare il progetto e la struttura di implementazione dei linguaggi di programmazione

L'importanza dei tipi anche nel Lambda Calcolo

- Abbiamo visto che il lambda calcolo ha la potenza espressiva delle macchine di Turing
- Possiamo codificare dati e strutture dati come espressioni del calcolo

$TRUE = \lambda t. \lambda f. t$
 $FALSE = \lambda t. \lambda f. f$

C_0 **$0 = \lambda s. \lambda z. z$**
 C_1 **$1 = \lambda s. \lambda z. s(z)$**

L'importanza dei tipi per il lambda calcolo

Dato che nel lambda calcolo i programmi e i valori sono funzioni, possiamo facilmente scrivere programmi che **non** sono **corretti** rispetto all'uso **inteso** dei valori

$$\textit{FALSE } 0 = (\lambda t. \lambda f. f)(\lambda s. \lambda z. z)$$

Qual è l'errore?

L'importanza dei tipi per il lambda calcolo

Dato che nel lambda calcolo i programmi e i valori sono funzioni, possiamo facilmente scrivere programmi che **non** sono **corretti** rispetto all'uso **inteso** dei valori

$$\textit{FALSE } 0 = (\lambda t. \lambda f. f)(\lambda s. \lambda z. z)$$

- ***FALSE*** si può effettivamente applicare a 0: produce un valore! (ma non ha senso nella nostra interpretazione di ***FALSE***...)
- I valori sono tutti rappresentati nello stesso modo (funzioni) ma **non si possono usare tutti nello stesso modo**

... è solo un problema del lambda calcolo?

La stessa cosa accade nel linguaggio macchina, in cui:

- le **istruzioni** sono parole della macchina: **sequenze di bit**
- i **dati** sono codificati come parole macchina: **sequenze di bit**
- le **operazioni** prendono in ingresso **sequenze di bit** e restituiscono come risultato **sequenze di bit**

Perché è importante controllare i tipi?

- Uno dei principi centrali dell'ingegneria del software è quello di realizzare opportuni **meccanismi** che permettano di ***rilevare precocemente errori di programmazione*** (ad esempio ***FALSE 0*** è un errore di programmazione)
- I linguaggi di programmazione supportano questa idea con i:
sistemi di tipo
(e più in generale con gli strumenti di **analisi statica**)

Che cosa sono i tipi?

- I **tipi** sono un modo per raggruppare i valori dei programmi in base al comportamento che vorremmo avessero
- Il **tipo** è un **attributo di un dato** che descrive come il linguaggio di programmazione permetta di usare quel particolare dato
 - I linguaggi di programmazione prevedono i tipi di dati di base come numeri interi (di varie dimensioni), numeri in virgola mobile (che approssimano i numeri reali), caratteri e booleani
- Un **tipo**
 - **limita i valori** che un'espressione, come una variabile o una funzione, può assumere
 - **definisce le operazioni** che possono essere fatte sui dati e il modo in cui i valori di quel tipo possono essere memorizzati

Esempi

- **3+2: *Int*** indica che ci si aspetta che
 - la computazione dia un risultato di tipo *Int*
 - l'operatore **+** venga usato correttamente, prendendo in ingresso due interi

Attenzione

- ***true: Int*** non va bene: *true* non è *Int*
- **2+true** non va bene: il **+** non è applicato bene

Esempio

Supponiamo che la seguente espressione sia inserita all'interno di un programma di grosse dimensioni

```
if (condizione_complicata)  
  { return "hello"/10; }
```

- Se i test del programma non forzano mai la valutazione di **<condizione_complicata>**, l'errore nel ramo then (**divisione di una stringa**) **non** verrà mai rilevato
- Tuttavia, un input non previsto nella batteria dei test potrebbe causare l'esecuzione del ramo "then", generando un errore a tempo di esecuzione
- Questo errore si sarebbe potuto evitare facilmente se il programmatore o l'implementatore del linguaggio avessero indicato l'espressione "**hello**"/10 come non appropriata

Passiamo a JavaScript

```
function addNumbers(x, y) {  
    return x + y;  
}  
// invocazione utente  
console.log(addNumbers(3, "0"));
```

Compone un numero
e una stringa



JavaScript Coercion:

l'operatore primitivo + può essere invocato con una coppia di valori di tipo (number, string)

Coercion: il valore numerico viene trasformato in un valore di tipo stringa

Nell'esempio viene stampato il valore "30"... una stringa

Passiamo a TypeScript

```
function addNumbers(x, y) {  
  return x + y;  
// invocazione utente  
console.log(addNumbers(3, "0"));
```

TypeScript: si accorge dell'errore e segnala un **errore di tipo**
rilevando questo **bug**
Come? Usando delle **regole di tipo**

Argument of type 'string'
is not assignable to
parameter of type 'number'

Sistema dei tipi

Un **sistema dei tipi** associa *tipi* ai valori calcolati.

Esaminando il flusso dei valori calcolati, il sistema dei tipi tenta di dimostrare che non avvengano ***errori di tipo***

Il sistema stesso determina **che cosa costituisce un *errore di tipo***, garantendo che le operazioni che si aspettano un certo tipo di valore non siano utilizzate con valori per i quali quell'operazione non ha senso

Sistema dei tipi

- Un sistema dei tipi è un **metodo sintattico**, **effettivo** e **strutturale** per **dimostrare l'assenza di comportamenti anomali** del programma, **strutturando** le operazioni del programma in base ai tipi di valori che calcolano

Sistema dei tipi

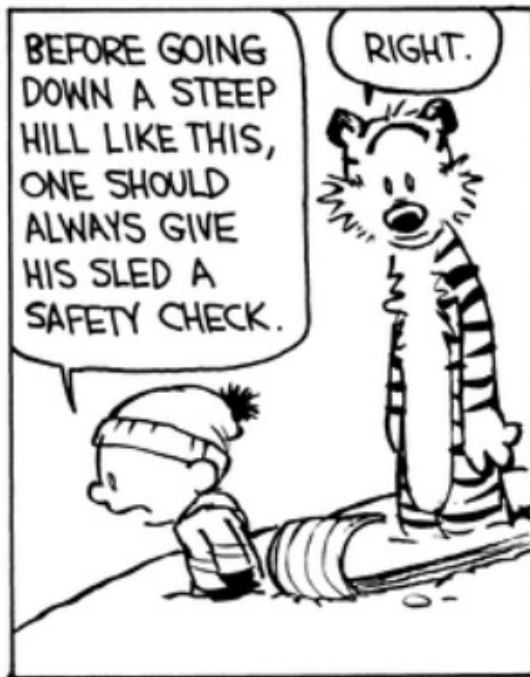
- Un sistema dei tipi è un **metodo sintattico**, **effettivo** e **strutturale** per **dimostrare l'assenza di comportamenti anomali** del programma, **strutturando** le operazioni del programma in base ai tipi di valori che calcolano
- **Sintattico**: la struttura sintattica guida il metodo di analisi del comportamento dei programmi

Sistema dei tipi

- Un sistema dei tipi è un **metodo sintattico**, **effettivo** e **strutturale** per **dimostrare l'assenza di comportamenti anomali** del programma, **strutturando** le operazioni del programma in base ai tipi di valori che calcolano
- **Sintattico**: la struttura sintattica guida il metodo di analisi del comportamento dei programmi
- **Effettivo**: si può definire **un algoritmo** che controlla i vincoli sui tipi e implementarlo in un compilatore o un interprete

Sistema dei tipi

- Un sistema dei tipi è un **metodo sintattico**, **effettivo** e **strutturale** per **dimostrare l'assenza di comportamenti anomali** del programma, **strutturando** le operazioni del programma in base ai tipi di valori che calcolano
- **Sintattico**: la struttura sintattica guida il metodo di analisi del comportamento dei programmi
- **Effettivo**: si può definire **un algoritmo** che controlla i vincoli sui tipi e implementarlo in un compilatore o un interprete
- **Strutturale**: i tipi assegnati alle componenti di un programma sono calcolati in modo **composizionale**, cioè il tipo di un'espressione dipende solo dai tipi delle sue sottoespressioni





Type safety

I tipi definiscono il comportamento sicuro (*safe behaviour*). Se possiamo dare un tipo a un'espressione possiamo usare il suo tipo per assicurarci che venga usata nel modo corretto

L'assenza di **type safety** (correttezza dell'uso dei tipi) porta a scrivere programmi pieni di **bug**

Implicazione in **cybersecurity**

I bug sono **vulnerabilità** del software: un attaccante può sfruttarli per alterare il comportamento del programma in modo malevolo o persino prendere il pieno controllo del flusso di esecuzione

Controllo dei tipi

- Il controllo può essere **statico** (in fase di compilazione) o **dinamico** (in fase di esecuzione), o una combinazione dei due
 - **Controllo statico**: trova gli errori prima di mandare il programma in esecuzione, non degrada le prestazioni
- Controllo di tipo (**type checker**) **verifica** che le **intenzioni del programmatore** (esprese dalle **annotazioni di tipo**) siano rispettate dal programma
- Se un programma supera il controllo dei tipi **è garantito che si comporti bene** in fase di esecuzione: non applicherà mai un'operazione a un valore di tipo non corretto
- **Controllo dei tipi: forma di correttezza parziale del programma (corretto rispetto ai tipi, e non rispetto ad altri tipi di errore)**

Domande significative

Come fa un progettista di linguaggio (o un programmatore) a sapere che programmi che superano il controllo dei tipi esibiscano poi a run-time le proprietà desiderate?

Per rispondere a questa domanda dobbiamo comprendere due aspetti essenziali:

- 1. come specificare i sistemi di tipi**
- 2. come dimostrare che un sistema di tipi è corretto**

- Per dimostrare la correttezza è indispensabile comprendere nel dettaglio il significato dei programmi (cioè cosa succede quando vengono eseguiti)**
- Comprendere i sistemi di tipo porterà a una comprensione più profonda del significato dei programmi**

Un primo caso di studio

Sistema dei tipi per espressioni

Espressioni e valori (sintassi)

Espressioni

E ::=

true

false

NV

if E then E else E

succ E

pred E

isZero E

Valori

V ::=

true

false

NV

Valori numerici

NV ::=

0 | 1 | 2 | ...

Regole di valutazione - semantica (interprete di espressioni)

Small-step semantics

if true then E_1 else $E_2 \rightarrow E_1$

IF-TRUE

if false then E_1 else $E_2 \rightarrow E_2$

IF-FALSE

$E \rightarrow E'$

if E then E_1 else $E_2 \rightarrow$ if E' then E_1 else E_2

IF-COND

Regole di valutazione - semantica

Small-step semantics

$$\frac{E \rightarrow E'}{\text{succ } E \rightarrow \text{succ } E'}$$

$$\frac{m = n + 1}{\text{succ } n \rightarrow m}$$

$$\frac{E \rightarrow E'}{\text{pred } E \rightarrow \text{pred } E'}$$

$$\frac{n > 0 \quad m = n - 1}{\text{pred } n \rightarrow m}$$

$$\text{pred } 0 \rightarrow 0$$

$$\frac{E \rightarrow E'}{\text{isZero } E \rightarrow \text{isZero } E'}$$

$$\text{isZero } 0 \rightarrow \text{true}$$

$$\frac{n > 0}{\text{isZero } n \rightarrow \text{false}}$$

Valutazione di *if isZero 0 then 2 else pred 1*

Valutazione di *if isZero 0 then 2 else pred 1*

isZero 0 → true

if true then E_1 else $E_2 \rightarrow E_1$

if false then E_1 else $E_2 \rightarrow E_2$

$E \rightarrow E'$

if E then E_1 else $E_2 \rightarrow$ if E' then E_1 else E_2

Valutazione di *if isZero 0 then 2 else pred 1*

if isZero 0 then 2 else pred 1 →

isZero 0 → *true*

if true then E₁ else E₂ → *E₁*

if false then E₁ else E₂ → *E₂*

E → *E'*

if E then E₁ else E₂ → *if E' then E₁ else E₂*

Valutazione di *if isZero 0 then 2 else pred 1*

isZero 0 → true

if isZero 0 then 2 else pred 1 →

isZero 0 → true

if true then E₁ else E₂ → E₁

if false then E₁ else E₂ → E₂

E → E'

if E then E₁ else E₂ → if E' then E₁ else E₂

Valutazione di *if isZero 0 then 2 else pred 1*

isZero 0 → true

if isZero 0 then 2 else pred 1 → if true then 2 else pred 1

isZero 0 → true

if true then E_1 else $E_2 \rightarrow E_1$

if false then E_1 else $E_2 \rightarrow E_2$

$E \rightarrow E'$

if E then E_1 else $E_2 \rightarrow if E' then E_1 else $E_2$$

Valutazione di *if isZero 0 then 2 else pred 1*

isZero 0 → true

if isZero 0 then 2 else pred 1 → if true then 2 else pred 1

if true then 2 else pred 1 → 2

isZero 0 → true

if true then E_1 else E_2 → E_1

if false then E_1 else E_2 → E_2

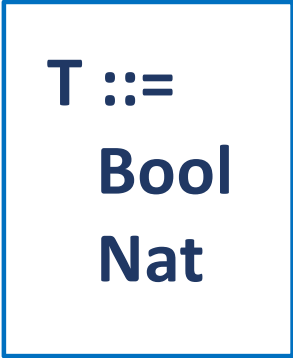
$E \rightarrow E'$

if E then E_1 else E_2 → if E' then E_1 else E_2

Tipi di espressioni

Il linguaggio delle espressioni prevede solo due **forme** sintattiche per i tipi: **Bool** o **Nat**

TIPI



T ::=
Bool
Nat

Controllo di tipo (type checker)

Il controllo di tipo definisce una relazione binaria

(E, T)

che associa il tipo T all'espressione E

Useremo la notazione

E:T

per indicare una coppia della relazione di assegnamento dei tipi

Controllo dei tipi

Metodo sintattico

**Regole definite per
induzione strutturale sulla
sintassi del programma**

Regole di controllo dei tipi

Insieme di regole di inferenza che assegnano tipi ai termini

$$\begin{array}{lll} & \frac{E: \text{Nat}}{\text{succ } E: \text{Nat}} & \\ \text{true: Bool} & & \\ \text{false: Bool} & \frac{E: \text{Nat}}{\text{pred } E: \text{Nat}} & \frac{E: \text{Bool}, E_1: T, E_2: T}{\text{if } E \text{ then } E_1 \text{ else } E_2: T} \\ n: \text{Nat} & & \\ & \frac{E: \text{Nat}}{\text{isZero } E: \text{Bool}} & \end{array}$$

Ogni espressione ha al più un tipo

Controllo di tipo (type checker)

- La **relazione di tipo** per le espressioni aritmetiche è la più piccola relazione binaria tra i termini e i tipi che soddisfa tutte le istanze delle regole viste
- Un termine **E** è **ben tipato** (*well-typed*) se esiste un **T** tale che

E:T

- Un termine può essere non ben tipato (*ill-typed*)

Controllo dei tipi: derivazione

Controllo dei tipi o **type checking**: E è del tipo T ?*

Dati E e T trovare una **derivazione**

$\frac{\text{???????}}{\textit{if isZero 0 then 2 else pred 1: Nat}}$

Ogni coppia (E, T) della relazione di tipo è caratterizzata da un **albero di derivazione** costruito da istanze delle regole di inferenza

*Invece **Type inference**: Trova un tipo T per l'espressione E

Data E , trovare T e una derivazione

Controllo dei tipi: derivazione

$$\frac{n: \text{Nat}}{E: \text{Nat}} \quad \frac{E: \text{Nat}}{\text{isZero } E: \text{Bool}} \quad \frac{E: \text{Nat}}{\text{pred } E: \text{Nat}} \quad \frac{E: \text{Bool}, E_1: T, E_2: T}{\text{if } E \text{ then } E_1 \text{ else } E_2: T}$$

isZero 0: Bool

if isZero 0 then 2 else pred 1: Nat

Ogni coppia (E, T) della relazione di tipo è caratterizzata da un **albero di derivazione** costruito da istanze delle regole di inferenza

Controllo dei tipi: derivazione

$$\frac{n: \text{Nat}}{E: \text{Nat}} \quad \frac{E: \text{Nat}}{\text{isZero } E: \text{Bool}} \quad \frac{E: \text{Nat}}{\text{pred } E: \text{Nat}} \quad \frac{E: \text{Bool}, E_1: T, E_2: T}{\text{if } E \text{ then } E_1 \text{ else } E_2: T}$$

$$\frac{\frac{0: \text{Nat}}{\text{isZero } 0: \text{Bool}}}{\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1: \text{Nat}}$$

Ogni coppia (E, T) della relazione di tipo è caratterizzata da un **albero di derivazione** costruito da istanze delle regole di inferenza

Controllo dei tipi: derivazione

$$\frac{n: \text{Nat}}{E: \text{Nat}} \quad \frac{E: \text{Nat}}{\text{isZero } E: \text{Bool}} \quad \frac{E: \text{Nat}}{\text{pred } E: \text{Nat}} \quad \frac{E: \text{Bool}, E_1: T, E_2: T}{\text{if } E \text{ then } E_1 \text{ else } E_2: T}$$

$$\frac{\frac{0: \text{Nat}}{\text{isZero } 0: \text{Bool}} \quad 2: \text{Nat} \quad \frac{}{\text{pred } 1: \text{Nat}}}{\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1: \text{Nat}}$$

Ogni coppia (E, T) della relazione di tipo è caratterizzata da un **albero di derivazione** costruito da istanze delle regole di inferenza

Controllo dei tipi: derivazione

$$\frac{}{n: \text{Nat}}$$
$$\frac{}{E: \text{Nat}}$$
$$\frac{}{\text{isZero } E: \text{Bool}}$$
$$\frac{}{E: \text{Nat}}$$
$$\frac{}{\text{pred } E: \text{Nat}}$$
$$\frac{}{E: \text{Bool}, E_1: T, E_2: T}$$
$$\frac{}{\text{if } E \text{ then } E_1 \text{ else } E_2: T}$$
$$\frac{\frac{0: \text{Nat}}{\text{isZero } 0: \text{Bool}} \quad 2: \text{Nat} \quad \frac{1: \text{Nat}}{\text{pred } 1: \text{Nat}}}{\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1: \text{Nat}}$$

Ogni coppia (E, T) della relazione di tipo è caratterizzata da un **albero di derivazione** costruito da istanze delle regole di inferenza

Dalla teoria all'algoritmo di typechecking (pseudocodice)

Le regole viste sono deterministiche

```
typeof(E) = if (E = true) then Bool
            else if (E = false) then Bool
            else if (E = if E1 then E2 else E3) then {
                T1 = typeof(E1); T2 = typeof(E2); T3 = typeof(E3)
                if (T1 = Bool and T2=T3) then T2
                else "type error" }
            else if (E = n) then Nat
            else if (E= succ E1) then {
                T1 = typeof(E1)
                if (T1 = Nat) then Nat else "type error" }
            else if (E = pred E1) then {
                T1 = typeof(E1)
                if (T1 = Nat) then Nat else "type error" }
            else if (E = iszero E1) then {
                T1 = typeof(E1)
                if T1 = Nat then Bool else "type error" }
```

true : Bool

false: Bool

E: Bool, E₁: T, E₂: T

if E then E₁ else E₂: T

n: Nat

E: Nat

succ E: Nat

E: Nat

pred E: Nat

E: Nat

isZero E: Bool

Precisione vs Approssimazione

I sistemi di tipo sono generalmente **imprecisi**: non definiscono esattamente quale tipo di valore sarà restituito da ogni programma, ma solo **un'approssimazione conservativa** o **sovrapprossimazione** considera tutte le esecuzioni possibili del programma ma anche alcune esecuzioni impossibili

Esempio

$$\frac{E: Bool, E_1: T, E_2: T}{if\ E\ then\ E_1\ else\ E_2: T}$$

Utilizzando la regola di tipo dell'IF **non** siamo in grado di associare un tipo all'espressione ***if true then 0 else false***

Questa espressione restituisce come risultato sicuramente un valore numerico: ma il sistema di tipo considera anche il caso del ramo else, impossibile a runtime

Composizionalità

Composizionalità Il tipo di un'espressione complessa dipende solo dalla sua struttura sintattica e dai tipi delle sue sotto-espressioni

Osservazione

La **regola** di controllo di tipo del **condizionale** richiede che le espressioni che costituiscono il “**ramo then**” e il “**ramo else**” **abbiano** lo stesso tipo, e che quel tipo sia il **tipo complessivo** del condizionale

$$\frac{E: Bool, E_1: T, E_2: T}{if\ E\ then\ E_1\ else\ E_2: T}$$

La regola garantisce la **composizionalità**. **Perché?**

Composizionalità (cont.)

Osservazione

La **regola** di controllo di tipo del **condizionale** richiede che le espressioni che costituiscono il “**ramo then**” e il “**ramo else**” **abbiano** lo stesso tipo, e che quel tipo sia il **tipo complessivo** del condizionale

$$\frac{E: Bool, E_1: T, E_2: T}{if\ E\ then\ E_1\ else\ E_2: T}$$

La regola garantisce la **composizionalità**. **Perché?**

Per essere usata in un certo **contesto**,
una **espressione** **E** deve avere
un **tipo** ben preciso

pred (if E then E₁ else E₂)
if (if E then E₁ else E₂) then E₃ else E₄

Composizionalità (cont.)

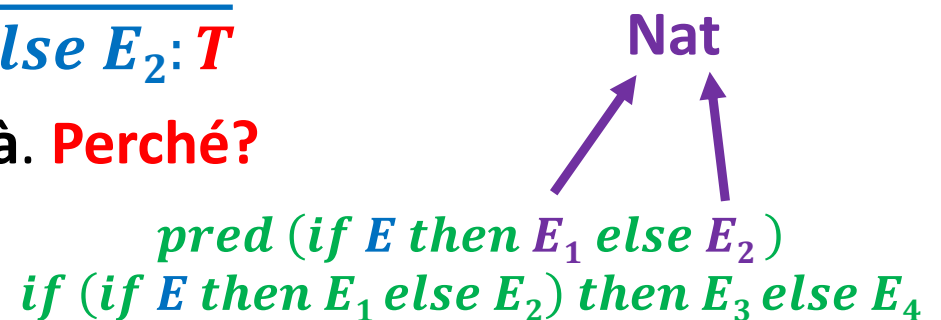
Osservazione

La **regola** di controllo di tipo del **condizionale** richiede che le espressioni che costituiscono il “**ramo then**” e il “**ramo else**” **abbiano** lo stesso tipo, e che quel tipo sia il **tipo complessivo** del condizionale

$$\frac{E: \text{Bool}, E_1: T, E_2: T}{\text{if } E \text{ then } E_1 \text{ else } E_2: T}$$

La regola garantisce la **composizionalità**. **Perché?**

Per essere usata in un certo **contesto**,
una **espressione** E deve avere
un **tipo** ben preciso



Qualunque espressione usata come

- operando di **pred** deve avere tipo **Nat**

Composizionalità (cont.)

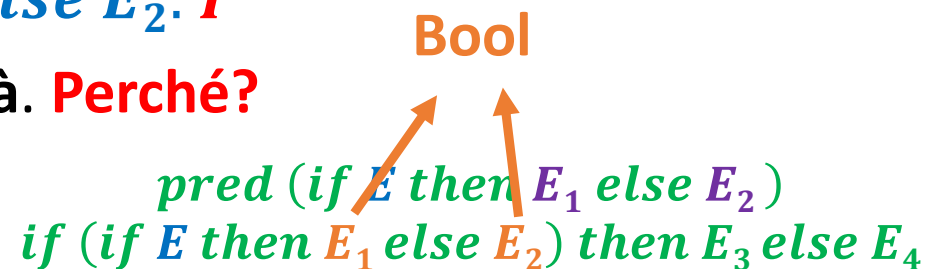
Osservazione

La **regola** di controllo di tipo del **condizionale** richiede che le espressioni che costituiscono il “**ramo then**” e il “**ramo else**” **abbiano** lo stesso tipo, e che quel tipo sia il **tipo complessivo** del condizionale

$$\frac{E: \text{Bool}, E_1: T, E_2: T}{\text{if } E \text{ then } E_1 \text{ else } E_2: T}$$

La regola garantisce la **composizionalità**. **Perché?**

Per essere usata in un certo **contesto**, una **espressione** E deve avere un **tipo** ben preciso



Qualunque espressione usata come

- operando di **pred** deve avere tipo **Nat**
- condizione di un **if** deve avere tipo **Bool**

Type Safety: Correttezza

La **correttezza** (*type safety* o *soundness*) del sistema di tipo è espressa formalmente da queste **due** proprietà e va dimostrata

Progresso: Se $E:T$ allora E è un valore oppure $E \rightarrow E'$ per una qualche espressione E'

Conservazione: Se $E:T$ e $E \rightarrow E'$ allora $E':T$

Well-typed terms do not "go wrong"

Type Safety: Correttezza

La **correttezza** (*type safety* o *soundness*) del sistema di tipo è espressa formalmente da queste due proprietà e va dimostrata

Progresso: Se $E:T$ allora E è un valore oppure $E \rightarrow E'$ per una qualche espressione E'

Se E è ben tipata non si blocca a run-time (può fare un passo)

Conservazione: Se $E:T$ e $E \rightarrow E'$ allora $E':T$

I tipi sono preservati dalle regole di esecuzione. Se E è ben tipata e fa un passo, anche il termine risultante è ben tipato

Type Safety: Correttezza

La **correttezza** (*type safety* o *soundness*) è una proprietà di un linguaggio di programmazione espressa formalmente da queste due proprietà:

Queste due proprietà, insieme, garantiscono che **l'intera esecuzione** dell'espressione non si bloccherà a run time durante la valutazione

Progresso: Se $E:T$ allora E è un valore oppure $E \rightarrow E'$ per una qualche espressione E'

Se E è ben tipata non si blocca a run-time (può fare un passo)

Conservazione: Se $E:T$ e $E \rightarrow E'$ allora $E':T$

I tipi sono preservati dalle regole di esecuzione. Se E è ben tipata e fa un passo, anche il termine risultante è ben tipato

Esempio

Progresso: Se $E:T$ allora E è un valore oppure $E \rightarrow E'$ per una qualche espressione E'

In caso di espressione non ben tipata, può succedere che il termine non sia un valore, ma non esista nemmeno una regola di valutazione che permetta di procedere

if 0 then 2 else 1 $\nrightarrow$

Esempio

Conservazione: Se $E:T$ e $E \rightarrow E'$ allora $E':T$

$$\frac{\frac{0: \text{Nat}}{\text{isZero } 0: \text{Bool}} \quad 2: \text{Nat} \quad \frac{1: \text{Nat}}{\text{pred } 1: \text{Nat}}}{\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1: \text{Nat}}$$

$\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1 \rightarrow \text{if true then } 2 \text{ else pred } 1$

$$\frac{\frac{\text{true}: \text{Bool}}{\text{true}: \text{Bool}} \quad 2: \text{Nat} \quad \frac{1: \text{Nat}}{\text{pred } 1: \text{Nat}}}{\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1: \text{Nat}}$$

$\text{if true then } 2 \text{ else pred } 1 \rightarrow 2$

$2: \text{Nat}$

$E' : T$

Progresso

Progresso: Se $E:T$ allora E è un valore oppure $E \rightarrow E'$ per una qualche espressione E'

Dimostrazione

Per induzione sulla struttura della derivazione di $E:T$

Casi base

I casi (T-True), (T-False) e (T-Nat) sono immediati, poiché in questi casi E è un valore (*true*, *false* e *n*)

true : Bool
false: Bool
n: Nat

Progresso

Progresso: Se $E:T$ allora E è un valore oppure $E \rightarrow E'$ per una qualche espressione E'

Dimostrazione

Per induzione sulla struttura della derivazione di $E:T$

Casi Induttivi

$E = \text{if } E_1 \text{ then } E_2 \text{ else } E_3:T$ (gli altri casi sono analoghi)

Applicando la regola IF_COND per il condizionale sappiamo che

$$E_1: \text{Bool}, E_2: T, E_3: T$$

Per ipotesi induttiva, (a) E_1 è un valore oppure (b) esiste E_4 tale che $E_1 \rightarrow E_4$

(a) Nel caso E_1 sia un valore allora deve essere necessariamente **true** o **false**

In questo caso possiamo fare il passo $E \rightarrow E_2$ oppure $E \rightarrow E_3$, rispettivamente

$$\frac{E_1: \text{Bool}, E_2: T, E_3: T}{E = \text{if } E_1 \text{ then } E_2 \text{ else } E_3: T}$$

if true then E_1 else $E_2 \rightarrow E_1$
if false then E_1 else $E_2 \rightarrow E_2$

Progresso

Progresso: Se $E:T$ allora E è un valore oppure $E \rightarrow E'$ per una qualche espressione E'

Dimostrazione

Per induzione sulla struttura della derivazione di $E:T$

Casi Induttivi

$E = \text{if } E_1 \text{ then } E_2 \text{ else } E_3:T$ (gli altri casi sono analoghi)

Applicando la regola IF_COND per il condizionale sappiamo che

$E_1: \text{Bool}, E_2: T, E_3: T$

$E_1: \text{Bool}, E_2: T, E_3: T$
 $E = \text{if } E_1 \text{ then } E_2 \text{ else } E_3:T$

$\text{if true then } E_1 \text{ else } E_2 \rightarrow E_1$
 $\text{if false then } E_1 \text{ else } E_2 \rightarrow E_2$
 $E \rightarrow E'$
 $\text{if } E \text{ then } E_1 \text{ else } E_2 \rightarrow \text{if } E' \text{ then } E_1 \text{ else } E_2$

Per ipotesi induttiva, (a) E_1 è un valore oppure (b) esiste E_4 tale che $E_1 \rightarrow E_4$

(a) Nel caso E_1 sia un valore allora deve essere necessariamente **true** o **false**

In questo caso possiamo fare il passo $E \rightarrow E_2$ oppure $E \rightarrow E_3$, rispettivamente

(b) Nel caso $E_1 \rightarrow E_4$ otteniamo che $\text{if } E_1 \text{ then } E_2 \text{ else } E_3 \rightarrow \text{if } E_4 \text{ then } E_2 \text{ else } E_3$

applicando la terza regola della semantica dell'IF, cioè E può fare un passo

Conservazione

Conservazione: Se $E:T$ e $E \rightarrow E'$ allora $E':T$

Dimostrazione Solita induzione strutturale sulla derivazione di $E:T$

Casi di base:

- se l'ultima regola applicata è **true: Bool** allora sappiamo dalla forma della regola che E è **true** e T deve essere **Bool**. Dato che **true** è un valore non si riduce e quindi la tesi è naturalmente vera
- Analogamente per gli altri casi base

Conservazione

Conservazione: Se $E:T$ e $E \rightarrow E'$ allora $E':T$

Dimostrazione Solita induzione strutturale sulla derivazione di $E:T$

Casi induttivi

Supponiamo che l'ultima regola applicata sia la regola IF_COND, sappiamo che

$E = \text{if } E_1 \text{ then } E_2 \text{ else } E_3:T$

Pertanto dobbiamo avere sotto-derivazioni con conclusioni

$E_1: \text{Bool}, E_2: T, E_3: T$

$$\frac{E_1: \text{Bool}, E_2: T, E_3: T}{E = \text{if } E_1 \text{ then } E_2 \text{ else } E_3: T}$$

Ci sono tre possibili regole per le quali $E \rightarrow E'$

1. IF-TRUE con $E_1 = \text{true}$ ed $E' = E_2$: abbiamo finito, dato che $E_2: T$
2. IF-FALSE con $E_1 = \text{false}$ ed $E' = E_3$: come prima, abbiamo finito, dato che $E_3: T$
3. IF-COND con $E_1 \rightarrow E'_1$ ed $E' = \text{if } E'_1 \text{ then } E_2 \text{ else } E_3$ dato che $E_1: \text{Bool}$ per ipotesi induttiva $E'_1: \text{Bool}$ quindi possiamo concludere che $E': T$

$$\frac{\text{true}: \text{Bool}, E_2: T}{E_2: T}$$

$$\frac{E'_1: \text{Bool}, E_2: T, E_3: T}{E' = \text{if } E'_1 \text{ then } E_2 \text{ else } E_3: T}$$

Conclusione

- Abbiamo visto come strutturare una strategia di definizione e di verifica di un sistema di tipo in un caso semplice: [il linguaggio delle espressioni](#)
- Passiamo ora a un secondo caso di studio: [il Lambda calcolo tipato](#)

Definizioni per le espressioni tipate

Sintassi

$$E ::= \text{true} \mid \text{false} \mid NV \mid \text{if } E \text{ then } E \text{ else } E \\ \mid \text{succ } E \mid \text{pred } E \mid \text{isZero } E$$
$$V ::= \text{true} \mid \text{false} \mid NV$$
$$NV ::= 0 \mid 1 \mid 2 \mid \dots$$

Tipi $T ::= \text{Bool} \mid \text{Nat}$

Regole di controllo dei tipi

$$\text{true} : \text{Bool} \quad \text{false} : \text{Bool} \quad n : \text{Nat}$$
$$\frac{E : \text{Nat}}{\text{succ } E : \text{Nat}} \quad \frac{E : \text{Nat}}{\text{pred } E : \text{Nat}} \quad \frac{E : \text{Nat}}{\text{isZero } E : \text{Bool}}$$
$$\frac{E : \text{Bool} \quad E1 : T \quad E2 : T}{\text{if } E \text{ then } E1 \text{ else } E2 : T}$$

Semantica (valutazione)

$$\text{if true then } E1 \text{ else } E2 \rightarrow E1$$
$$\text{if false then } E1 \text{ else } E2 \rightarrow E2$$
$$\frac{E \rightarrow E'}{\text{if } E \text{ then } E1 \text{ else } E2 \rightarrow \text{if } E' \text{ then } E1 \text{ else } E2}$$
$$\frac{E \rightarrow E'}{\text{succ } E \rightarrow \text{succ } E'} \quad \frac{m = n + 1}{\text{succ } n \rightarrow m}$$
$$\frac{E \rightarrow E'}{\text{pred } E \rightarrow \text{pred } E'} \quad \frac{n > 0 \quad m = n - 1}{\text{pred } n \rightarrow m}$$
$$\text{pred } 0 \rightarrow 0 \quad \text{isZero } 0 \rightarrow \text{true}$$
$$\frac{E \rightarrow E'}{\text{isZero } E \rightarrow \text{isZero } E'} \quad \frac{n > 0}{\text{isZero } n \rightarrow \text{false}}$$

Il Lambda calcolo tipato

Lambda calcolo tipato semplice

- Come abbiamo visto a lezione, il lambda-calcolo nella sua forma "pura" non prevede tipi
- Per comodità è più semplice avere un insieme di tipi di base, quindi, a rigore, nella letteratura, esistono numerose varianti del lambda calcolo tipato semplice a seconda della scelta dei tipi base
- Noi consideriamo inizialmente una variante costruita sui soli **valori booleani** (oltre che su valori di tipo **funzione**, già visti)

#1: Sintassi dei tipi

$\tau ::=$	Tipi
Bool	Tipo dei booleani
$\tau \rightarrow \tau$	Tipo delle funzioni

Scegliere come tipo $\rightarrow$
non sarebbe troppo utile

Esempi di tipi **sintatticamente corretti**:

Bool Bool $\rightarrow$ Bool (Bool $\rightarrow$ Bool) $\rightarrow$ Bool
 (Bool $\rightarrow$ Bool) $\rightarrow$ (Bool $\rightarrow$ Bool)

Attenzione: Il costruttore di tipo $\rightarrow$ associa a **destra**: $\tau_1 \rightarrow (\tau_2 \rightarrow \tau_3)$

Bool $\rightarrow$ Bool $\rightarrow$ Bool sta per Bool $\rightarrow$ (Bool $\rightarrow$ Bool)

#2: Sintassi del linguaggio

FUN: Lambda calcolo tipato semplice con booleani

e ::=	Espressioni
x	Variabili
fun x: τ = e	Funzioni
Apply(e, e)	Applicazione
true	Costante true
false	Costante false
if e then e else e	Condizionale

Attenzione questa è l'**espressione condizionale** non il comando condizionale:
i due rami contengono espressioni e non comandi

#2: Sintassi del linguaggio (formato tradizionale)

FUN: Lambda calcolo tipato semplice con booleani

$e ::=$

x

$\lambda x:\tau. e$

$e e$

true

false

if e then e else e

Espressioni

Variabili

Funzioni

Applicazione

Costante true

Costante false

Condizionale

#2: Valori

$V ::=$

$\text{fun } x:\tau = e$

true

false

Valori

Funzioni

Valore true

Valore false

#3: Semantica (cose già viste... ma riscritte con la nuova sintassi)

$$\text{Apply } (\text{fun } x:\tau = e_1, v) \rightarrow e_1\{x := v\}$$

$$\frac{e_1 \rightarrow e'}{\text{Apply}(e_1, e_2) \rightarrow \text{Apply}(e', e_2)} \qquad \frac{e_2 \rightarrow e'}{\text{Apply}(v, e_2) \rightarrow \text{Apply}(v, e')}$$

$$\frac{e_1 \rightarrow e_4}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \rightarrow \text{if } e_4 \text{ then } e_2 \text{ else } e_3} \quad \text{if true then } e_2 \text{ else } e_3 \rightarrow e_2$$
$$\text{if false then } e_2 \text{ else } e_3 \rightarrow e_3$$

#3: Semantica (cose già viste... ma riscritte con la nuova sintassi)

$$\frac{e_1 \rightarrow e'}{\text{Apply}(e_1, e_2) \rightarrow \text{Apply}(e', e_2)}$$

$$\frac{e_2 \rightarrow e'}{\text{Apply}(v, e_2) \rightarrow \text{Apply}(v, e')}$$

β -riduzione con strategia
call-by-value

$$\frac{\text{if } e_1 \text{ then } e_2 \text{ else } e_3}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3}$$

$$\frac{\text{if true then } e_2 \text{ else } e_3 \rightarrow e_2 \quad \text{if false then } e_2 \text{ else } e_3 \rightarrow e_3}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3}$$

espressione condizionale

#4: Composizionalità del type checker

Definiamo le regole del type checker induttivamente sulla struttura sintattica del linguaggio

Problema:

$$\lambda x:\tau. e \quad \frac{x:\tau_1 \quad \text{?????}}{\text{fun } x:\tau_1 = e:\tau_1 \rightarrow \tau_2}$$

#4: Composizionalità del type checker

Definiamo le regole del type checker induttivamente sulla struttura sintattica del linguaggio

Problema:

$$\lambda x:\tau. e \quad \frac{x:\tau_1 \quad \text{?????}}{\text{fun } x:\tau_1 = e:\tau_1 \rightarrow \tau_2}$$

- Ci aspettiamo che il tipo del risultato della funzione sia il tipo del corpo e
- Il tipo del corpo e della funzione dipende dal tipo del parametro formale x che dipende dal tipo che gli è stato assegnato al momento della definizione
- Come teniamo conto di questa associazione?

#3: tipi delle variabili

Ambiente dei tipi (*type environment*)

L'ambiente dei tipi è una funzione (di **dominio finito**) che associa nomi a tipi.
Noi scriveremo:

$$\Gamma = x_1:\tau_1, x_2:\tau_2 \dots x_k:\tau_k$$

per indicare la funzione

$$\Gamma(x_i) = \tau_i$$

che associa il tipo τ_i al valore x_i

La notazione

$$\boxed{\Gamma, x:\tau}$$

sarà usata per indicare l'**estensione** della funzione Γ con l'associazione $x:\tau$

$$(\Gamma, x:\tau)(y) = \begin{cases} \tau & \text{per } y = x \\ \Gamma(y) & \text{per } y \neq x \end{cases}$$

Esempio

- Se abbiamo come ambiente $\Gamma = x:\tau, y:\tau'$ allora
 - $\Gamma(x) = ?$
 - $\Gamma(z) = ?$

Esempio

- Se abbiamo come ambiente $\Gamma = x:\tau, y:\tau'$ allora
 - $\Gamma(x) = \tau$
 - $\Gamma(z) = \textit{undefined}$

Esempio

- Se abbiamo come ambiente $\Gamma = x:\tau, y:\tau'$ allora
 - $\Gamma(x) = \tau$
 - $\Gamma(z) = \textit{undefined}$
- Se abbiamo come ambiente $\Gamma' = \Gamma, z:\tau_2$ allora
 - $\Gamma'(z) = ?$

Esempio

- Se abbiamo come ambiente $\Gamma = x:\tau, y:\tau'$ allora
 - $\Gamma(x) = \tau$
 - $\Gamma(z) = \textit{undefined}$
- Se abbiamo come ambiente $\Gamma' = \Gamma, z:\tau_2$ allora
 - $\Gamma'(z) = \tau_2$

Esempio

L'**ambiente dei tipi vuoto** $\emptyset$ non contiene alcun legame di tipo per le variabili

$\emptyset(x) = \textit{undefined}$
per tutte le variabili x

Giudizio di tipo

Supponiamo che Γ sia un ambiente di tipo.

Si usa la notazione

$$\Gamma \vdash e : \tau$$

Siamo passati da una relazione di tipo a 2 posti ($e : \tau$) a una relazione a 3 posti

per indicare che l'espressione e ha tipo τ nell'ambiente di tipo Γ

INTUIZIONE: L'ambiente dei tipi tiene conto dei legami tra i nomi che compaiono nel programma (l'espressione e) e il loro tipo

Giudizio di tipo

Supponiamo che Γ sia un ambiente di tipo.

Si usa la notazione

$$\Gamma \vdash e : \tau$$

per indicare che l'espressione e ha tipo τ nell'ambiente di tipo Γ

- Le **regole** sono applicate **dal compilatore in fase di analisi statica**
- L'**ambiente dei tipi** nel gergo dei compilatori è chiamato **Tabella dei Simboli (Symbol Table)**

Regole di tipo

Definiamo il sistema per il controllo dei tipi (type checker) per il nostro Lambda calcolo tipato semplice

$$\Gamma \vdash \textit{true} : \textit{Bool}$$

$$\Gamma \vdash \textit{false} : \textit{Bool}$$

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}$$

Una variabile ha il tipo che le è associato nell'ambiente dei tipi Γ
La **premessa** si legge come
Se si assume che x abbia tipo τ

Condizionale

$$\frac{\Gamma \vdash e : \textit{Bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \textit{if } e \textit{ then } e_1 \textit{ else } e_2 : \tau}$$

- In un'espressione condizionale, i **due rami** devono avere lo **stesso tipo τ** che è poi quello complessivo
- nella regola, il tipo τ può essere istanziato con qualsiasi tipo di funzione, permettendoci di avere condizionali i cui rami sono funzioni

Tipi per le funzioni

- Quale è il tipo di una funzione?
- Il costruttore di tipo $\tau_1 \rightarrow \tau_2$ descrive il tipo delle funzioni che prendono in ingresso un argomento di tipo τ_1 e restituiscono un risultato di tipo τ_2

Funzioni

$$\frac{\Gamma, x: \tau_1 \vdash e: \tau_2}{\Gamma \vdash \text{fun } x: \tau_1 = e: \tau_1 \rightarrow \tau_2}$$

Dato che il tipo τ_1 del parametro formale x è noto:

- (i) il **tipo** delle occorrenze del parametro x nel corpo della funzione saranno associate a τ_1
- (ii) il **tipo** del **risultato** della funzione sarà il **tipo** del **corpo** della funzione

Intuizione

La funzione richiede un parametro di tipo τ_1 e restituisce come risultato un valore di tipo τ_2

Notazione: $\tau_1 \rightarrow \tau_2$

Funzioni

$$\frac{\Gamma, x: \tau_1 \vdash e: \tau_2}{\Gamma \vdash \text{fun } x: \tau_1 = e: \tau_1 \rightarrow \tau_2}$$

Intuizione (informatica)

- La definizione del parametro formale x e del suo tipo è una specie di dichiarazione dinamica
- La dichiarazione del parametro x sovrascrive e annulla (nell'ambiente esteso $\Gamma, x: \tau_1$) le eventuali precedenti dichiarazioni per x (che sono in Γ)
- La portata (scope) della dichiarazione del parametro x è il corpo della funzione e :
- Infatti $\Gamma, x: \tau_1$ viene usato solo per tipare il corpo della funzione

Esempio

$$\frac{\emptyset, x: \textit{Bool} \vdash x: \textit{Bool}}{\emptyset \vdash \textit{fun } x: \textit{Bool} = x: \textit{Bool} \rightarrow \textit{Bool}}$$

Chiamata di funzione

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash \text{Apply}(e_1, e_2) : \tau_2}$$

- Se e_1 è una funzione che mappa argomenti in τ_1 a risultati in τ_2 (le cui variabili libere hanno i tipi assunti in Γ), e
- se la valutazione di e_2 ha tipo τ_1 ,
- allora il risultato dell'applicazione di e_1 a e_2 sarà un valore di tipo τ_2

Esempio

$\emptyset \vdash \textit{fun } f: \textit{Bool} \rightarrow \textit{Bool} = \textit{Apply}(f, \textit{if false then true else false}): (\textit{Bool} \rightarrow \textit{Bool}) \rightarrow \textit{Bool}$

Esempio

$$\frac{\Gamma(f) = Bool \rightarrow Bool}{\Gamma \vdash f : Bool \rightarrow Bool}$$

$$\emptyset \vdash \textit{fun } f: Bool \rightarrow Bool = \textit{Apply}(f, \textit{if false then true else false}): (Bool \rightarrow Bool) \rightarrow Bool$$

Esempio

$$\frac{\Gamma(f) = Bool \rightarrow Bool}{\Gamma \vdash f : Bool \rightarrow Bool} \quad \frac{\Gamma \vdash false: Bool \quad \Gamma \vdash true: Bool \quad \Gamma \vdash false: Bool}{\Gamma \vdash if\ false\ then\ true\ else\ false: Bool}$$

$$\emptyset \vdash \textit{fun } f: Bool \rightarrow Bool = \textit{Apply}(f, if\ false\ then\ true\ else\ false): (Bool \rightarrow Bool) \rightarrow Bool$$

Esempio

$$\frac{\frac{\Gamma(f) = Bool \rightarrow Bool}{\Gamma \vdash f : Bool \rightarrow Bool} \quad \frac{\Gamma \vdash false: Bool \quad \Gamma \vdash true: Bool \quad \Gamma \vdash false: Bool}{\Gamma \vdash if\ false\ then\ true\ else\ false: Bool}}{\Gamma = f: Bool \rightarrow Bool \vdash Apply(f, if\ false\ then\ true\ else\ false): Bool}$$

$$\frac{}{\emptyset \vdash \textit{fun } f: Bool \rightarrow Bool = Apply(f, if\ false\ then\ true\ else\ false): (Bool \rightarrow Bool) \rightarrow Bool}$$

Type Safety: Correttezza

La **correttezza** (*type safety*) del sistema di tipo del lambda calcolo è espressa formalmente da queste due proprietà e va dimostrata

Progresso: Se $\emptyset \vdash e : \tau$ allora e è un valore oppure $e \rightarrow e'$ per una qualche espressione e'

Se e non ha variabili libere ed è ben tipata non si blocca a run-time

Conservazione: Se $\Gamma \vdash e : \tau$ ed $e \rightarrow e'$ allora $\Gamma \vdash e' : \tau$

I tipi sono preservati dalle regole di esecuzione. Se e è ben tipata e fa un passo, anche il termine risultante è ben tipato

Teorema del progresso

Progresso: Se $\emptyset \vdash e : \tau$ allora e è un valore oppure $e \rightarrow e'$ per una qualche espressione e'

Come si dimostra?

Per induzione sulle derivazioni di tipo

- **Casi di base (costanti booleane):** immediati e identici ai casi di base del sistema per le espressioni aritmetiche (e' è sempre un valore)
- **Caso delle variabili:** banale, perché una variabile non è ben tipata nell'ambiente vuoto (e deve essere un termine chiuso)
- **Caso dell'astrazione funzionale:** immediato, poiché le funzioni sono valori

Teorema del progresso

Progresso: Se $\emptyset \vdash e : \tau$ allora e è un valore oppure $e \rightarrow e'$ per una qualche espressione e'

Casi induttivi (solo il caso dell'applicazione)

Applicazione: $e = \text{Apply}(e_1, e_2)$, $\emptyset \vdash e_1 : \tau_1 \rightarrow \tau_2$ $\emptyset \vdash e_2 : \tau_1$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash \text{Apply}(e_1, e_2) : \tau_2}$$

Per l'ipotesi induttiva possiamo affermare che

e_1 è un valore o può fare un passo di valutazione, e così pure e_2

- Se e_1 può fare un passo, applicando la regola
- Analogamente se e_1 è un valore ed e_2 può fare un passo, con
- Se sono entrambi valori, allora abbiamo che e_1 deve essere

$$\frac{e_1 \rightarrow e'}{\text{Apply}(e_1, e_2) \rightarrow \text{Apply}(e', e_2)}$$

$$\frac{e_2 \rightarrow e'}{\text{Apply}(v, e_2) \rightarrow \text{Apply}(v, e')}$$

$$\text{fun } x : \tau_1 = e' : \tau_1 \rightarrow \tau_2$$

Pertanto può fare un passo, applicando la regola $\text{Apply}(\text{fun } x : \tau = e_1, v) \rightarrow e_1\{x := v\}$

Teorema della conservazione

Conservazione: Se $\Gamma \vdash e : \tau$ ed $e \rightarrow e'$ allora $\Gamma \vdash e' : \tau$

Si dimostra sempre per induzione strutturale sulle derivazioni di tipo.
Quale è il caso difficile?

Teorema della conservazione

Conservazione: Se $\Gamma \vdash e : \tau$ ed $e \rightarrow e'$ allora $\Gamma \vdash e' : \tau$

Si dimostra sempre per induzione strutturale sulle derivazioni di tipo.

Quale è il caso difficile?

Applicazione: $e = \text{Apply}(e_1, e_2)$

Quindi vale che

$$\begin{array}{c} \Gamma \vdash e : \tau \\ \Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \\ \Gamma \vdash e_2 : \tau_1 \\ \tau = \tau_2 \\ e \rightarrow e' \end{array}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash \text{Apply}(e_1, e_2) : \tau_2}$$

Si deve pertanto dimostrare che

$$\Gamma \vdash e' : \tau_2$$

Teorema della conservazione

Conservazione: Se $\Gamma \vdash e: \tau$ ed $e \rightarrow e'$ allora $\Gamma \vdash e': \tau$

Vale

$$\Gamma \vdash e: \tau$$

$$\Gamma \vdash e_1: \tau_1 \rightarrow \tau_2$$

$$\Gamma \vdash e_2: \tau_1$$

$$\tau = \tau_2$$

$$e \rightarrow e'$$

Si deve dimostrare che

$$\Gamma \vdash e': \tau_2$$

$\Gamma \vdash e_1: \tau_1 \rightarrow \tau_2$ comporta che:

$$e_1 = \text{fun } x: \tau_1 = e_3$$

$$\Gamma, x: \tau_1 \vdash e_3: \tau_2$$

$$\frac{\Gamma, x: \tau_1 \vdash e: \tau_2}{\Gamma \vdash \text{fun } x: \tau_1 = e: \tau_1 \rightarrow \tau_2}$$

$$e' = e_3 \{x := e_2\}$$



$$\text{Apply } (\text{fun } x: \tau = e_1, v) \rightarrow e_1 \{x := v\}$$

ATTENZIONE: abbiamo dimostrato $e_3: \tau_2$ e invece la semantica dell'applicazione ci porta in $e_3 \{x = e_2\}$, non in e_3 ... si deve perciò gestire opportunamente la **sostituzione [che non deve cambiare il tipo]**

Substitution lemma

Substitution Lemma: I tipi sono preservati dall'operazione di sostituzione
Se $\Gamma, x: \tau_1 \vdash e: \tau$ e $\Gamma \vdash e_1: \tau_1$ allora $\Gamma \vdash e\{x := e_1\}: \tau$

Come si dimostra?

Per **induzione sulla derivazione** di

$$\Gamma, x: \tau_1 \vdash e: \tau$$

I casi più interessanti sono quelli per le variabili e le astrazioni funzionali
(La dimostrazione completa si trova sulle note didattiche)

Intuizione sostituire un parametro formale con un parametro attuale dello stesso tipo, fa sì che si preservi il tipo dell'espressione

Esempio: se $\text{if } x \text{ then } e_1 \text{ else } e_2: \text{Nat}$ nell'ipotesi che $x: \text{Bool}$ allora
 $(\text{if } x \text{ then } e_1 \text{ else } e_2)\{x := \text{true}\}: \text{Nat}$



Estensioni (cenni)

Abbiamo considerato un
lambda calcolo tipato con
costanti booleane e
condizionale

Possiamo estendere il
linguaggio con altri costrutti
per farlo diventare un vero
linguaggio di
programmazione

Sintassi

FUN: Lambda calcolo tipato semplice con booleani, naturali e operazioni numeriche

e ::=	Espressioni
x	Variabili
fun x: τ = e	Funzioni
Apply(e, e)	Applicazione
true	Costante true
false	Costante false
n	Costanti numeriche
e $\oplus$ e	Operazioni binarie
if e then e else e	Condizionale

n ::= 0, 1, 2

$\oplus$::= + - * / %

Si assume che siano noti i tipi delle operazioni primitive $\oplus: \tau_1 \times \tau_2 \rightarrow \tau$

Regole di tipo

$$\Gamma \vdash n : \mathit{Nat}$$

$$\frac{\Gamma \vdash e_1 : \tau_1, \Gamma \vdash e_2 : \tau_2 \quad \oplus : \tau_1 \times \tau_2 \rightarrow \tau}{\Gamma \vdash e_1 \oplus e_2 : \tau}$$

Dichiarazioni locali

$e ::= \dots$

$\text{let } x = e_1 \text{ in } e_2 : \tau_2$

Regole di valutazione

$$\frac{e_1 \rightarrow e'}{\text{let } x = e_1 \text{ in } e_2 : \tau_2 \rightarrow \text{let } x = e' \text{ in } e_2 : \tau_2}$$
$$\text{let } x = v \text{ in } e_2 : \tau_2 \rightarrow e_2\{x := v\} : \tau_2$$

Dichiarazioni locali

$e ::= \dots$

$\text{let } x = e_1 \text{ in } e_2 : \tau_2$

Regola di tipo

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2}$$

Ricorsione

Si può facilmente dimostrare che nel lambda calcolo tipato semplice che abbiamo introdotto tutti i **programmi tipabili terminano**

Notare che il combinatore Ω che abbiamo introdotto non è tipabile e nemmeno Y lo è

Il prossimo passo è estendere il linguaggio con un costrutto linguistico per gestire la **ricorsione** che sia tipabile (con tutte le proprietà che ci aspettiamo)

$$\begin{aligned}\Omega &= (\lambda x. x x)(\lambda x. x x) \\ Y &= \lambda f. (\lambda x. f(x x))(\lambda x. f(x x))\end{aligned}$$

L'auto-applicazione non è tipabile

$$\frac{\Gamma \vdash x: \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash x: \tau_1}{\Gamma \vdash \textit{Apply}(x, x): \tau_2}$$

Dato che x può avere un tipo soltanto, ne segue che

τ_1 dovrebbe essere uguale a $\tau_1 \rightarrow \tau_2$

Verso la ricorsione con un esempio

```
let G = fun f: Nat → Nat =  
    fun n: Nat =  
        if (isZero n) then 1 else Times (n f (pred n))
```

$G: (Nat \rightarrow Nat) \rightarrow Nat \rightarrow Nat$

Verso la ricorsione con un esempio

```
G =  $\lambda f. \lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * f(n - 1)$ 
```

```
let G = fun f: Nat → Nat =  
    fun n: Nat =  
        if (isZero n) then 1 else Times (n f (pred n))
```

G: (Nat → Nat) → Nat → Nat

Aggiungiamo al linguaggio un **costrutto fix** tale che:

- fact = **fix** G

Applicando fix a G si ottiene il limite delle approssimazioni... ovvero il **punto fisso**

I passi verso la definizione di fix

Sintassi espressioni

$e ::= \dots \mid \text{fix } e$

fix e : definizione ricorsiva

Regole di valutazione

$$\frac{e \rightarrow e'}{\text{fix } e \rightarrow \text{fix } e'}$$

Regole di tipo

$$\frac{\Gamma \vdash e : \tau \rightarrow \tau}{\Gamma \vdash \text{fix } e : \tau}$$

$$\text{fix } (\text{fun } f : \tau = e) \rightarrow e[f := (\text{fix } (\text{fun } f : \tau = e))]$$

$$\text{fix}(\lambda x : \tau. e) \rightarrow e[x := \text{fix}((\lambda x : \tau. e))]$$

Fattoriale

$G = \lambda f. \lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * f(n - 1)$

let G = fun f: Nat → Nat =
 fun n: Nat =
 if (isZero n) then 1 else Times (n f (pred n))

G: (Nat → Nat) → Nat → Nat

$\text{fix}(\lambda x: \tau. e) \rightarrow e[x := \text{fix}(\lambda x: \tau. e)]$

Esempio di applicazione

fact 3 = (fix G) 3 = (fix ($\lambda f. \lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * f(n - 1)$)) 3 →
($\lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * f(n - 1)$)[f := fix G] 3 =
($\lambda n. \text{if } (n = 0) \text{ then } 1 \text{ else } n * \text{fix G}(n - 1)$) 3 →
(if (3 = 0) then 1 else 3 * fix G(3 - 1)) →* 3 * fix G(3 - 1)

I passi verso la definizione di fix

Lambda calcolo tipato semplice + fix = PCF
Programming Computable Functions (PCF)
è un linguaggio funzionale tipato, introdotto
da Gordon Plotkin nel 1977

Sintassi espressioni

$e ::= \dots \mid \text{fix } e$

Regole di valutazione

$$\frac{e \rightarrow e'}{\text{fix } e \rightarrow \text{fix } e'}$$

$$\frac{}{\text{fix } (\text{fun } f: \tau = e) \rightarrow e[f := (\text{fix } (\text{fun } f: \tau = e))]}$$

Regole di tipo

$$\frac{\Gamma \vdash e: \tau \rightarrow \tau}{\Gamma \vdash \text{fix } e: \tau}$$

Una sintassi più semplice (zucchero sintattico)

let rec $x:\tau = e$ *in* e'

corrisponde a *let* $x = \text{fix } (\text{fun } x:\tau = e)$ *in* e'

In OCaml

```
let rec fact: Nat → Nat =
```

```
  fun n: Nat =
```

```
    if (isZero n) then 1 else Times (n f (pred n))
```

```
in fact 3;
```

Ricorsione in OCaml: osservazione

- Nel linguaggio OCaml come vedremo tra poco abbiamo **due** primitive per definire funzioni
- La **primitiva fun** permette di definire **funzioni non ricorsive**
 - `(fun x -> x + 1)`
 - `let plusOne = fun x -> x+1;;`
- La **primitiva let rec** permette di definire **funzioni ricorsive**
 - **let rec** fact x =
 if x <= 1 then 1 else x * fact (x - 1)

The background is a solid black field. On the left side, there are two horizontal bands of light orange color, separated by a black band featuring a white zigzag pattern. A small light green circle is positioned on the left edge of the central black rectangle. In the bottom right corner, there is a large, light green semi-circular shape. The central black rectangle is outlined with a thin white border and a slightly thicker light orange border.

CONSIDERAZIONI

Conclusioni

- Il lambda calcolo è un modello fondazionale essenziale per comprendere la teoria della computazione
- È utile per capire i linguaggi di programmazione: sistemi dei tipi, terminazione, sistemi di verifica, etc. possono essere analizzati nel lambda calcolo per poi scalare a linguaggi di programmazione completi

Definizioni per il λ -calcolo tipato

Sintassi $V ::= true \mid false \mid fun\ x : \tau = e$ $e ::= x \mid fun\ x : \tau = e \mid Apply(e, e) \mid true \mid false$ $\quad \mid if\ e\ then\ e\ else\ e$	Tipi $\tau ::= Bool \mid \tau \rightarrow \tau$
Regole di controllo dei tipi $\frac{\Gamma \vdash true : Bool \quad \Gamma \vdash false : Bool \quad \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}}{\Gamma \vdash E : Bool \quad \Gamma \vdash E1 : \tau \quad \Gamma \vdash E2 : \tau}$ $\frac{\Gamma \vdash E : Bool \quad \Gamma \vdash E1 : \tau \quad \Gamma \vdash E2 : \tau}{\Gamma \vdash if\ E\ then\ E1\ else\ E2 : \tau}$ $\frac{\Gamma, x : \tau \vdash e : \tau_1 \rightarrow \tau_2}{\Gamma \vdash fun\ x : \tau_1 = e : \tau_1 \rightarrow \tau_2}$ $\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash Apply(e_1, e_2) : \tau_2}$	Semantica (valutazione) $if\ true\ then\ e_1\ else\ e_2 \rightarrow e_1$ $if\ false\ then\ e_1\ else\ e_2 \rightarrow e_2$ $\frac{e_1 \rightarrow e_4}{if\ e_1\ then\ e_2\ else\ e_3 \rightarrow if\ e_4\ then\ e_2\ else\ e_3}$ $Apply(fun\ x : \tau = e_1, v) \rightarrow e_1\{x := v\}$ $\frac{e_1 \rightarrow e'}{Apply(e_1, e_2) \rightarrow Apply(e', e_2)}$ $\frac{e_2 \rightarrow e'}{Apply(e_1, e_2) \rightarrow Apply(v, e_2)}$

- I **tipi** sono **specifiche di comportamento**: il sistema dei tipi che abbiamo considerato specifica il comportamento input-output delle funzioni e il type checking verifica l'adeguatezza del comportamento input-output
- Tipi e type checking possono essere usati per verificare **proprietà di programmi**
 - proprietà di **segretezza e autenticità** dei protocolli di sicurezza
 - proprietà comportamentali (**assenza di deadlock**) in sistemi concorrenti

Tipi e Ingegneria del Software

- Il type checking fornisce delle garanzie di **correttezza parziale** e inevitabilmente **rifiuta alcuni programmi corretti**
- La maggior parte delle **proprietà interessanti non possono essere automaticamente verificate** quindi i tipi possono solo dare un'approssimazione della correttezza

Proprietà di programmi e decidibilità

- Alcune proprietà interessanti:
 - Il programma termina su tutti in dati in ingresso?
 - Quanto grande può diventare la memoria dinamica (heap)?
 - Viene garantita la privacy dell'informazione?
- Le proprietà che coinvolgono la previsione esatta del comportamento del programma **sono generalmente indecidibili** (Teorema di Rice, 1953)
- Non possiamo semplicemente eseguire il programma e vedere cosa succede, perché non c'è un limite superiore al tempo di esecuzione dei programmi

Teorema di Rice

**Proprietà non banali del
comportamento di programmi
non sono decidibili**

CLASSES OF RECURSIVELY ENUMERABLE SETS AND THEIR DECISION PROBLEMS⁽¹⁾

BY

H. G. RICE

1. Introduction. In this paper we consider classes whose elements are recursively enumerable sets of non-negative integers. No discussion of recursively enumerable sets can avoid the use of such classes, so that it seems desirable to know some of their properties. We give our attention here to the properties of complete recursive enumerability and complete recursiveness (which may be intuitively interpreted as decidability). Perhaps our most interesting result (and the one which gives this paper its name) is the fact that no nontrivial class is completely recursive.

We assume familiarity with a paper of Kleene [5]⁽²⁾, and with ideas which are well summarized in the first sections of a paper of Post [7].

I. FUNDAMENTAL DEFINITIONS

2. Partial recursive functions. We shall characterize recursively enumer-

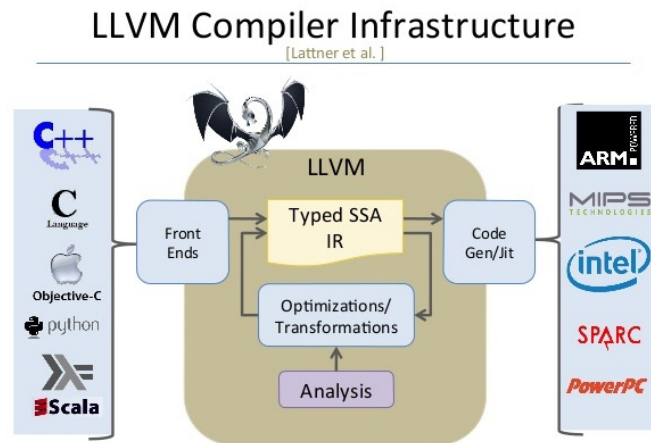
La soluzione adottata nei sistemi di tipo

Le soluzioni approssimate al problema possono essere decidibili!

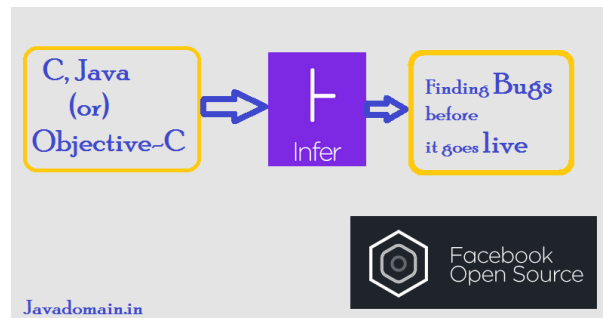
-> si possono definire sistemi di tipi per le proprietà da studiare

L'approssimazione deve preservare i comportamenti (teorema del progresso e della conservazione)

ESEMPI DI ANALIZZATORI STATICI (BASATI SU SISTEMI DI TIPI E APPROCCI SIMILI):



F
L
O
W
D
R
O
I
D



INFER



ROSLYN CODE ANALYZER